

1 UART TX

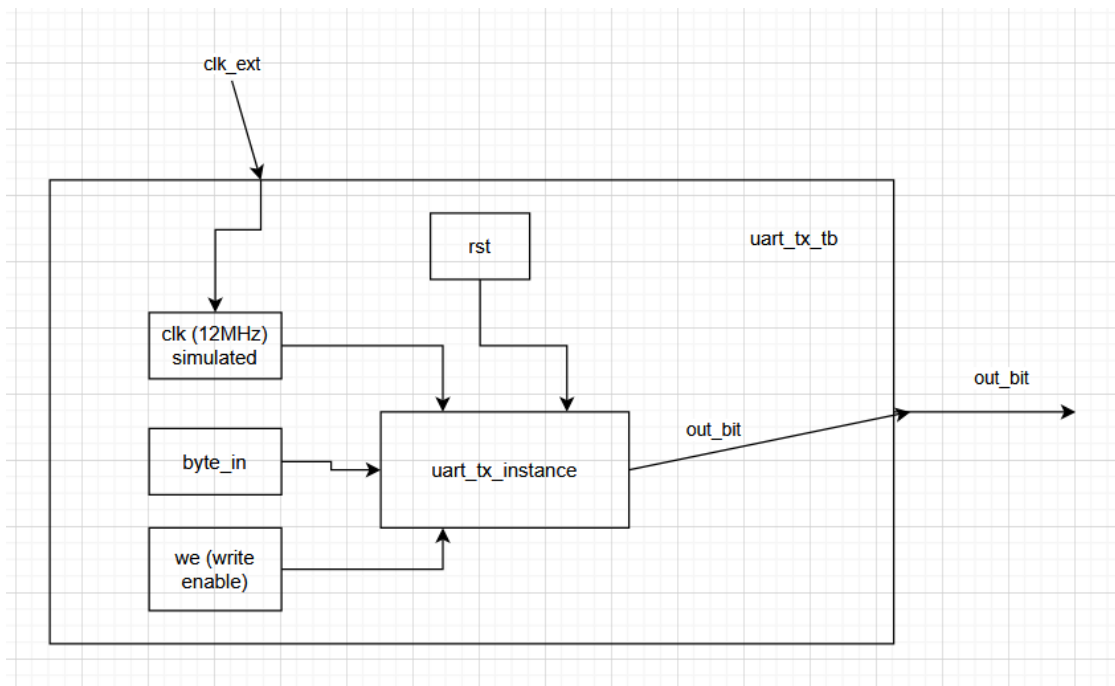


Abbildung 1: UART TX Blockdiagramm

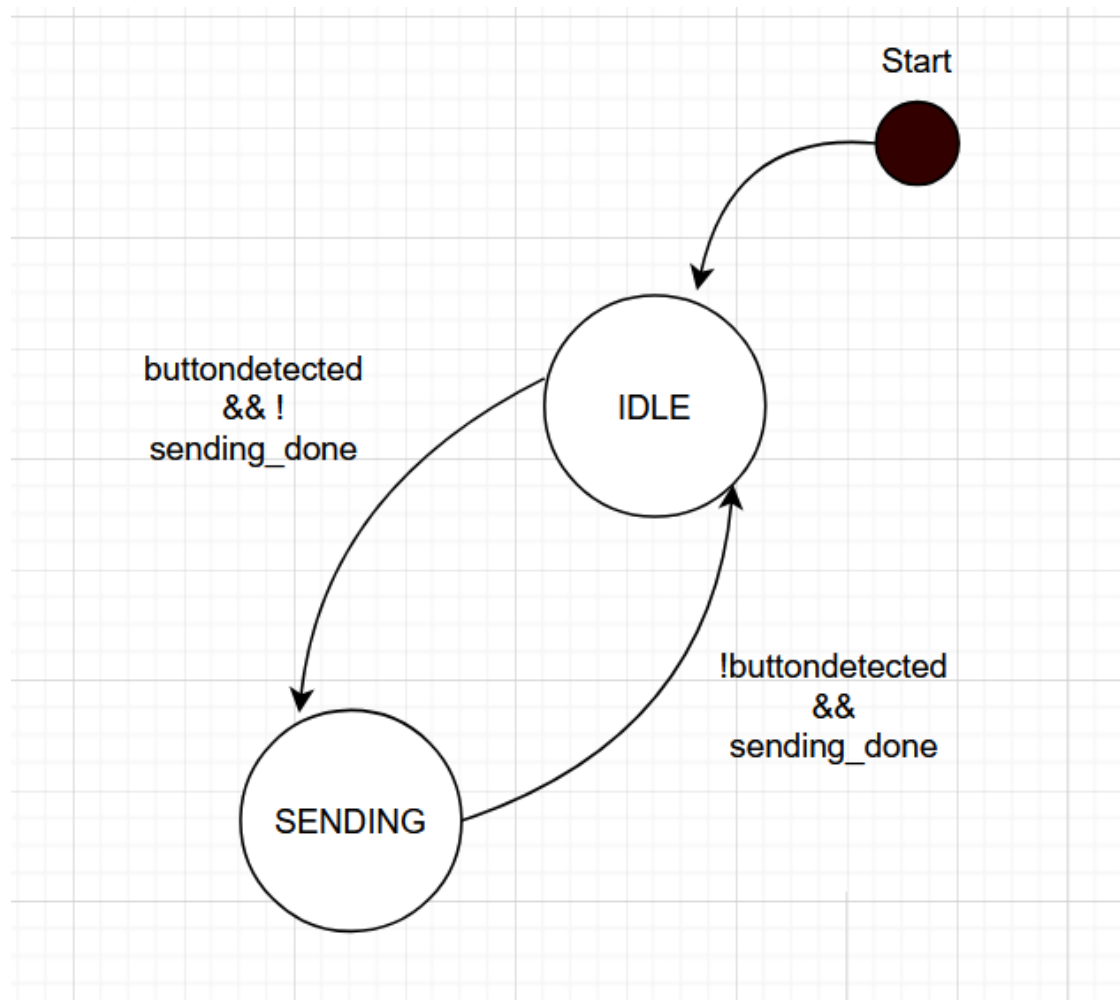


Abbildung 2: UART TX Statusmaschine

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_tx is
6      port(
7          clk : in std_logic;
8          byte_in : in std_logic_vector(7 downto 0);
9          we : in std_logic;
10         out_bit : out std_logic;
11         rst : in std_logic
12     );
13     end entity;
14
15
16     architecture uart_tx_a of uart_tx is
17         type state_type is (IDLE, SENDING);
18         signal state : state_type := IDLE;
19         signal symbol_cnt : integer := 0;
20         constant baud_rate : integer := 9600;
21         constant clk_speed : integer := 12000000;
22         constant bit_period : integer := clk_speed / baud_rate;
23         signal sending_done : std_logic := '0';
24         signal button_detected : std_logic := '0';
25
26     begin
27         process(clk) is
28             variable cnt_period : integer := 0;
29             variable start_bit_done : std_logic := '0';
30             variable char_bits_done : std_logic := '0';
31             begin
32                 if rising_edge(clk) then
33
34                     if rst = '1' then
35                         state <= IDLE;
36                         out_bit <= '1';
37                     else
38
39                         if button_detected = '1' then
40                             state <= SENDING;
41                             button_detected <= '0';
42                         elsif sending_done = '1' then
43                             state <= IDLE;
44                             sending_done <= '0';
45                         end if;
46
47                         case state is
48                             when IDLE =>
49                                 if we = '1' then
50                                     button_detected <= '1';

```

```

51         else
52             out_bit <= '1';
53         end if;
54         when SENDING =>
55             if start_bit_done = '0' then
56                 out_bit <= '0';
57                 if cnt_period < bit_period then
58                     cnt_period := cnt_period + 1;
59                 else
60                     cnt_period := 0;
61                     start_bit_done := '1';
62                 end if;
63             elsif char_bits_done = '1' then
64                 out_bit <= '1';
65                 if cnt_period < bit_period then
66                     cnt_period := cnt_period + 1;
67                 else
68                     cnt_period := 0;
69                     char_bits_done := '0';
70                     start_bit_done := '0';
71                     sending_done <= '1';
72                 end if;
73             else
74                 if symbol_cnt < 8 then
75
76                     out_bit <= byte_in(symbol_cnt);
77                     cnt_period := cnt_period + 1;
78
79                     if cnt_period > (bit_period - 1 ) then
80                         symbol_cnt <= symbol_cnt + 1;
81                         cnt_period := 0;
82                     end if;
83                 else
84                     symbol_cnt <= 0;
85                     char_bits_done := '1';
86                 end if;
87             end if;
88
89         end case;
90
91     end if;
92
93     end if;
94 end process;
95 end architecture uart_tx_a;

```

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_tx_tb is
6  end entity;
7
8
9  architecture uart_tx_tb_a of uart_tx_tb is
10     signal clk : std_logic := '0';
11     signal byte_in : std_logic_vector(7 downto 0) := (others => '0');
12     signal we : std_logic := '0';
13     signal rst : std_logic := '0';
14     signal out_bit : std_logic := '0';
15
16 begin
17     uart_tx_inst : entity work.uart_tx
18     port map(
19         clk=> clk,
20         byte_in => byte_in,
21         we => we,
22         rst => rst,
23         out_bit => out_bit
24     );
25
26     clk <= not clk after 83.33 ns / 2;
27
28     process
29     begin
30         -- Reset
31         rst <= '1';
32         wait for 100 ns;
33         rst <= '0';
34
35         -- Check IDLE
36         wait for 50 ns;
37         assert out_bit = '1' report "ERROR: out_bit not idle high after reset"
38             ↪ severity error;
39
40         -- 1
41         byte_in <= "11010001";
42         wait for 100 ns;
43         we <= '1';
44         wait for 100 ns;
45         we <= '0';
46
47         wait for 50 us;
48         assert out_bit = '0'
49             report "ERROR: Startbit not detected (Test 1)"
50                 severity warning;

```

```

50
51     wait for 2 ms;
52     assert out_bit = '1'
53         report "ERROR: Line not back to idle (Test 1)"
54             severity error;
55
56     -- 2
57     byte_in <= "10101010";
58     wait for 100 ns;
59     we <= '1';
60     wait for 300 ns;
61     we <= '0';
62
63     wait for 50 us;
64     assert out_bit = '0'
65         report "ERROR: Startbit not detected (Test 2)"
66             severity warning;
67
68     wait for 2 ms;
69
70     -- 3
71     byte_in <= "11110000";
72     wait for 100 ns;
73     we <= '1';
74     wait for 300 ns;
75     we <= '0';
76
77     wait for 50 us;
78     assert out_bit = '0'
79         report "ERROR: Startbit not detected (Test 3)"
80             severity warning;
81
82     wait for 2 ms;
83
84     -- 4
85     byte_in <= "00001111";
86     wait for 100 ns;
87     we <= '1';
88     wait for 300 ns;
89     we <= '0';
90
91     wait for 50 us;
92     assert out_bit = '0'
93         report "ERROR: Startbit not detected (Test 4)"
94             severity warning;
95
96     wait for 2 ms;
97
98     -- 5
99     byte_in <= "00110011";

```

```

100     wait for 100 ns;
101     we <= '1';
102     wait for 300 ns;
103     we <= '0';
104
105     wait for 50 us;
106     assert out_bit = '0'
107         report "ERROR: Startbit not detected (Test 5)"
108             severity warning;
109
110     wait for 2 ms;
111
112     -- Ende
113     assert false report "Simulation finished successfully" severity note;
114     wait;
115
116
117     end process;
118 end architecture uart_tx_tb_a;

```